Problems and solutions in AresOS

Real bugs vs. false positives, the scheduler, and the all-process model

Operating Systems – ITBA

1 Objective and method

An analysis tool suggested a list of bugs and improvements. This document audits each item against the real code and classifies it as a **real bug** (fixed), a **false positive** (intentional or correct behavior), or a **deferred / out-of-scope improvement**. It then documents the larger design changes that followed: the scheduler, the mvar app, the conversion of every command into a process, and the removal of the last busy-wait. For every change we give the root cause, the fix, and the empirical validation in QEMU.

The criterion to separate a real bug from a false positive is conservative: only something that produces an observable incorrect state (deadlock, corruption, resource loss) under a reachable execution is classified as a bug.

2 Bug A: race condition in `process_wait`

2.1 Diagnosis

The tool flagged a window between the loop test and the parent blocking. The analysis confirms the bug is **real**, and the root cause is in the `syscall` configuration: the FMASK MSR is initialized to zero.

```
mov ecx, 0xC0000084   ; IA32_FMASK
xor eax, eax
xor edx, edx
wrmsr                 ; FMASK = 0: syscall masks no RFLAGS bit
```

With FMASK = 0, the interrupt flag (IF) stays enabled throughout the syscall handler. A timer tick can therefore preempt `process_wait` between testing the child's state and the parent's transition to `BLOCKED`. The original code wrote `BLOCKED` with interrupts on:

```
while (target->state != PROCESS_ZOMBIE && target->state != PROCESS_DEAD) {
        current->waiting_for = pid;
        current->state       = PROCESS_BLOCKED;  // <- a tick here loses the wakeup
        scheduler_yield();
        _hlt();
}
```

2.2 Sequence that produces the deadlock

Step	Event
1	The parent evaluates the condition: the child is still alive, it enters the body.
2	A timer tick preempts before `state = BLOCKED`.
3	The scheduler runs the child; the child calls `process_exit` and then `wake_waiters`.
4	`wake_waiters` only wakes processes already in `BLOCKED`; the parent is still `RUNNING`, so it does not match. The child becomes `ZOMBIE`.
5	The parent resumes, sets itself `BLOCKED`, and sleeps. Nobody will wake it again: deadlock.

Evidence that the window is reachable: `sem_wait` already guarded this exact pattern with `cli`, while `process_wait` did not.

2.3 Fix

The child-state test and the transition to `BLOCKED` are done atomically with interrupts off, mirroring the discipline already used by `sem_wait`:

```
for (;;) {
        uint64_t flags = irq_save();
        if (target->pid != pid || target->state == PROCESS_ZOMBIE ||
            target->state == PROCESS_DEAD) {
                irq_restore(flags);
                break;
        }
        current->waiting_for = pid;
        current->state       = PROCESS_BLOCKED;
        irq_restore(flags);

        scheduler_yield();
        while (current->state == PROCESS_BLOCKED)
                _hlt();
}
```

If the child exits inside the critical section, the next loop iteration reads `ZOMBIE` and exits; if it exits afterwards, `wake_waiters` finds the parent already `BLOCKED` and wakes it. The lost-wakeup window is gone.

2.4 Validation

In QEMU, running `div 6 2` in the foreground prints `6 / 2 = 3` and returns the prompt: the shell waits with `process_wait`, reaps the child, and continues. Across a multi-command session the shell never hung.

3 Bug B: the exception handler did not terminate the process

3.1 Diagnosis

A **real bug** for a multiprocess kernel. The `exceptionHandler` macro rewrote the return address with the userland entry and did `iretq`, restarting the shell regardless of which process faulted and without freeing its resources:

```
popState
call getStackBase
mov [rsp+24], rax     ; new RSP = stack base
mov rax, userland
mov [rsp], rax        ; new RIP = 0x400000 (restarts userland)
sti
iretq
```

If a process other than the shell faulted, the other PCBs were left inconsistent and the faulting process's resources leaked.

3.2 Fix

The C dispatcher terminates the faulting process when it is not the shell. Since `process_exit` never returns (the scheduler reaps the zombie on the next tick), the macro's tail – the `iretq` to `0x400000` – is only reached for the shell, which is still recovered in place:

```
if (pid != SHELL_PID)
        process_exit(KILLED_EXIT_CODE);
```

This path reuses the normal process exit (cleanup of pipes, semaphores, stacks), so it introduces no new hazards.

3.3 On-screen feedback

Besides the register dump, after an exception the user is told which process faulted and what the kernel did about it, for feedback and debugging. The message is printed after the dump (not before) so it survives the screen being cleared on overflow:

```
ncPrint("\nFaulting process: ", VGA_CYAN);
ncPrint(pcb != NULL ? pcb->name : "unknown", VGA_WHITE);
ncPrint(" (pid ", VGA_CYAN); ncPrintDec((uint64_t)pid); ncPrint(")\n", VGA_CYAN);
// Action: process terminated by the kernel; the rest of the system keeps running.
```

3.4 Validation

`div 1 0` and `opcode`, now run as processes, trigger exceptions 0 and 6 with user selectors (`CS = 0x23`, `SS = 0x1B`), confirming the fault happens in ring
3. The screen shows the dump, the line `Faulting process: div (pid N)`, and

`Action: process terminated by the kernel`. On a key press, the shell returns to the prompt without restarting.

4 Bug C: divide-by-zero in `div` (false positive)

The tool flagged that `div` does not validate the divisor. This is a **false positive**: the `div` command exists precisely to demonstrate the exception-0 handler, and validating the divisor would remove the only test vector.

The legitimate concern behind the report – that the exception "breaks the user's flow" – is solved at the root by Bug B plus making `div` a process: today `div 1 0` faults in an isolated process that the kernel kills cleanly, without affecting the shell or the rest of the system. The demo was not removed.

5 Everything is a process

The system already supported processes, so the built-in/process distinction was removed entirely: the shell (`sh`, PID 0) resolves every command by name and spawns it as a process. There are no built-ins left.

Command logic that touches shell-private state (the command table, the history buffer, the cursor shape, the shell start time) lives in `commands.c` and is called by thin app wrappers in `apps.c`; because the whole userland shares one flat address space, a spawned process reads/writes those globals directly.

Group	Commands (now all processes)
Required apps	`mem`, `ps`, `loop`, `kill`, `nice`, `block`, `cat`, `wc`, `filter`, `mvar`, plus the shell `sh`

Group	Commands (now all processes)
Former built-ins	`help`, `man`, `time`, `clear`, `history`, `inforeg`, `cursor`, `textcolor`, `bgcolor`, `exit`
Architecture-TP apps	`div`, `opcode`, `tron`, `printmem`, `benchmark`

Direct benefits: a fault in any command is contained to its process (Bug B), and any command can be backgrounded or piped (e.g. `help | wc`). The shell's dispatch collapses to "spawn by name", removing the typed-lambda built-in machinery.

6 Priority as scheduling frequency

6.1 Why the quantum scheme was not enough

The original scheduler was round robin with `quantum = priority` (a higher-priority process runs more consecutive ticks). That makes priority observable for **CPU-bound** work (`test_prio` passes), but not for **cooperative** work that yields the CPU: a process that yields never uses its longer quantum, so in `mvar` raising a writer's priority changed nothing.

6.2 Fix: deficit round robin

Priority is now scheduling **frequency**. Each round a process is eligible to be picked `priority` times (a per-PCB credit), so a higher-priority process is selected proportionally more often. Credits refill once every ready process has spent theirs.

```
static int pick_next_ready(void) {
        int idx = scan_ready_with_credit();   // next READY with credit > 0
        if (idx != NO_READY_PROCESS) return idx;
        refill_credits();                     // new round: credits = priority
        return scan_ready_with_credit();
}
```

This makes priority observable for both CPU-bound and yield-bound workloads.

6.3 Cooperative yield: immediate context switch

For the frequency to matter in a yielding workload, each `yield()` must give up the CPU immediately rather than wait for the next timer tick (the old `sys_yield` did `scheduler_yield(); _hlt();`, a fixed one-tick floor that masked priority). A dedicated software vector `0x81` re-enters the scheduler now, without timekeeping or PIC EOI:

```
_irq81Handler:                ; software-triggered context switch
        pushState
        mov rdi, rsp
        call do_yield_switch  ; reschedule without advancing the clock
        mov rsp, rax
        popState
        iretq                 ; adapts to the saved frame (ring 0 if parked
                              ; mid-syscall, ring 3 if it came from userland)
```

Because `iretq` is privilege-aware, a process parked mid-syscall (ring-0 frame) and a freshly created or timer-preempted process (ring-3 frame) are both resumed by the same `popState + iretq`; the scheduler only stores/loads the saved `rsp`.

6.4 Validation

`test_prio` finishes in priority order (the priority-4 process first, then 2, then 1; equal-priority processes finish together). In `mvar 2 1`, `nice`-ing a writer up makes its letter dominate the output (runs of `BBB`). The course tests (`test_mm`, `test_proc`, `test_sync`) and the blocking paths keep working.

7 The `mvar` application

`mvar` is a single-cell MVar: `empty`/`full` semaphores, writers do `wait(empty) / write / post(full)`, readers do `wait(full) / consume / post(empty)`. This matches the Haskell MVar semantics (one value at a time, single-consumer, FIFO fairness). Two refinements were made to keep that behaviour faithful:

- **Active wait by yielding**: the random active wait is a loop of `yield()` calls (cooperative) instead of a CPU busy-wait. It does not hog the CPU (so no process is starved) and, combined with frequency-based priority, makes `nice` observable: a higher-priority writer counts its yields down faster and reaches the variable more often.
- **Per-process RNG**: the course `GetUniform()` keeps one static state shared by every process, so all mvar workers drew from the same interleaved stream and their waits became correlated, making the output erratic. Each writer/reader now seeds its own Marsaglia MWC generator with its pid, giving independent random streams.

The exact output is non-deterministic by design (it depends on scheduling and the random waits); the assignment's table is a representative output, not a literal one. What holds are the invariants: mutual exclusion, one consumer per value, unique letter per writer, unique color per reader, the main process exiting immediately, and `nice`/`kill` changing the pattern as expected.

8 No busy-waiting: blocking sleep in the sound driver

The assignment requires the system to be free of busy-waiting except where a test needs it (`loop`, `test_sync` without semaphores). The only remaining violation was the sound driver, which spun while a note played:

```
static void busy_wait_ms(uint64_t ms) {       // removed
        uint64_t end = get_time_ms() + ms;
        while (get_time_ms() < end) ;
}
```

It was replaced by a real blocking sleep: the calling process is parked (`BLOCKED` with a deadline) and the timer wakes it once the deadline passes, while other processes run.

```
void process_sleep_ms(uint64_t ms) {
        pcb_t *self = process_get_current();
        uint64_t flags = irq_save();
        self->sleep_until_ms = get_time_ms() + ms;
        self->state          = PROCESS_BLOCKED;
        irq_restore(flags);
        while (self->state == PROCESS_BLOCKED)
                _hlt();              // no spinning
}
```

`schedule()` calls `process_wake_sleepers(get_time_ms())` each tick to wake processes whose deadline has passed. `playSound` now does `enable_speaker(); process_sleep_ms(duration);`

`disable_speaker();`. Validated by running `tron`, whose start/collision/game-over melodies (sequences of timed notes) play through to completion without hanging.

9 Reaping orphans and re-parenting children on death

9.1 Diagnosis

The scheduler reaps a finished process only when it is the **outgoing** one: in `do_schedule` it calls `reap_if_zombie(current)`, which frees a zombie's PCB and stacks once nobody waits on it. A zombie is never scheduled again, so this check only ever sees the process that just stopped running. That leaves a gap: a process that is left as a zombie **pending reap by a waiter**, whose waiter then dies before reaping it, is never re-evaluated and leaks its PCB and stacks forever.

The sequence that exposes it, using `test_proc` (which creates dummy children and `wait`s on the ones it kills):

Step	Event
1	`test_proc` kills a dummy child while waiting on it, so `process_has_waiter(child)` is true: the child is left `ZOMBIE` for `test_proc` to reap.
2	`test_proc` itself is killed before it reaps the child.
3	Now no live process waits on the child, but it is not the running process either, so nothing ever reaps it: it stays `ZOMBIE` and its memory is lost.

The same shape appears with `mvar`: the launcher process spawns the writers and readers and exits immediately, so those workers were left **orphaned** (their parent PCB dead) the instant the launcher was reaped. They kept running and printing to the console – output goes to stdout, which is independent of parenthood – but the parent-child accounting was already inconsistent.

This is a bounded scheduler bug, not a fundamental limitation: the cause is that reaping looks only at the outgoing process and never at zombies that become orphaned afterwards.

9.2 Fix

A single helper, invoked from both `process_exit` and `process_kill` when a process dies, re-parents the survivors and reaps the orphans:

```
static void reparent_and_reap_orphans(pid_t dead_pid) {
        for (int i = 0; i < MAX_PROCESSES; i++) {        // living children: init-
style
                pcb_t *child = &process_table[i];         // re-parent to the shell
                if (child->parent_pid == dead_pid && child->pid != dead_pid &&
                    child->state != PROCESS_DEAD)
                        child->parent_pid = SHELL_PID;
        }
        for (int i = 0; i < MAX_PROCESSES; i++) {        // zombies nobody waits on:
                pcb_t *zombie = &process_table[i];        // reap now (skip the
running
                if (zombie->state == PROCESS_ZOMBIE &&    // one, the scheduler reaps
it)
                    zombie->pid != current_pid &&
                    !process_has_waiter(zombie->pid)) {
                        process_free_resources(zombie->pid);
                        zombie->state = PROCESS_DEAD;
                }
```

```
        }
}
```

Living children are handed to the shell (`SHELL_PID`), the init of this system, so they are never orphaned: they keep running and remain waitable and killable. Any zombie that no longer has a waiter is reaped immediately, except the running process, which cannot free its own stack and is reaped by the scheduler on the next tick as before. The helper subsumes the previous "kill with no waiter, reap now" branch of `process_kill`.

9.3 Validation

Running `test_proc 5 &`, then killing it mid-execution:

State	`Used` (bytes)	Stuck zombie?
Before the fix, after `kill`	184976	yes (a `ZOMBIE` child lingered)
After the fix, after `kill`	—	no (children re-parented to the shell)
After the fix, children also killed	50848	no (back to baseline)

Killing the re-parented survivors brings `Used` back to the exact baseline of 50848 bytes (8 live blocks), so nothing is leaked. The normal paths still hold: a divide-by-zero exception (which exits through `process_exit`) returns to baseline, and `test_sync` run repeatedly settles at a constant 50944 bytes – the one-time growth of the semaphore slab cache, which retains freed slabs for reuse by design, not a leak.

Memory a process requests for itself through the `malloc` syscall is reclaimed too. `sys_malloc` prepends a 16-byte list node to every user allocation (so the returned pointer stays 16-byte aligned) and links it into a per-process list anchored in the PCB; `sys_free` unlinks and frees it, and `process_free_resources` frees whatever is left when the process dies. So killing `test_mm` – the one application that exercises the `malloc` syscall – mid-loop returns `Used` to the exact baseline instead of leaking the blocks it was holding, on both the first-fit and the buddy allocator (the tracking lives at the syscall layer, so it is allocator-agnostic).

10 Deferred / out-of-scope items

Suggestion	Status	Rationale
Real scroll in video	deferred	The console clears on overflow. A genuine UX improvement, not a requirement.
Sleep instead of busy-wait in sound	done	Implemented as a blocking sleep (see section above).
Paging / memory protection	out of scope	The model is flat with identity mapping by design. Not a bug; very high effort.
Multiple pipes	deferred	A single pipe meets the requirement. Generalizing the parsing is an optional extension.

11 Conclusion

Of the three reported bugs, two were real (`A` and `B`) and were fixed with empirical validation; the third (`C`) was a false positive whose underlying concern was neutralized by making commands processes. On top of that, the scheduler was reworked so priority is observable for cooperative workloads, every command became a process, `mvar` was refined to stay faithful to MVar semantics, the last busy-wait (the sound driver) was removed, and process death now reaps orphan zombies

and re-parents living children so killing a process leaks no kernel resources. The system has no known deadlock in `wait`, isolates faults per process, reclaims a process's own memory whether it exits or is killed, demonstrates priority in both CPU-bound and cooperative workloads, and is free of busy-waiting outside the cases the assignment allows.